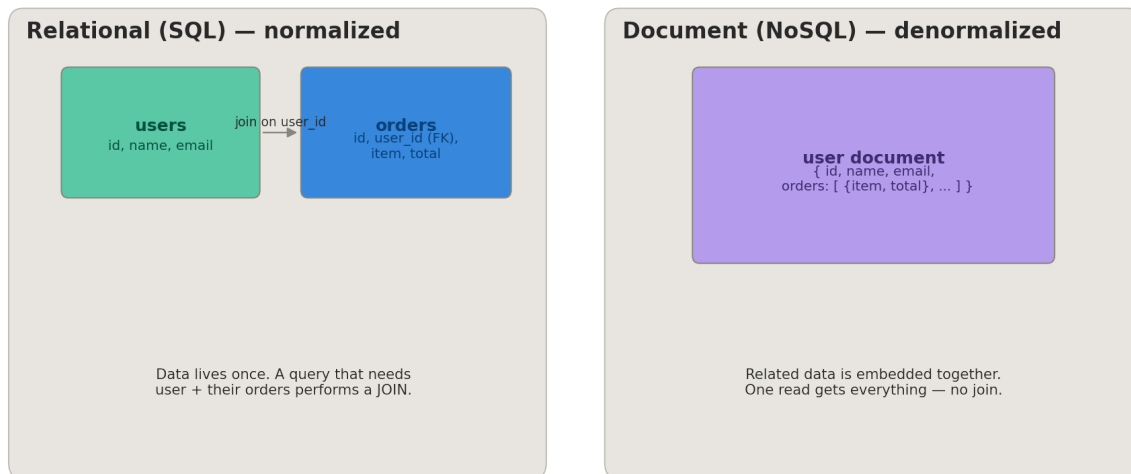


Stage 3 Crash Course: Data Storage

This is the stage where most system design interviews are actually won or lost. Almost every case study reduces, eventually, to “how do you store and access this data at scale.” Take this one slowly.

1. SQL vs. NoSQL

This is not “which is better” — it's “what access pattern and consistency need forces which.”



- SQL (relational): data is normalized — each fact lives in exactly one place, and related data is joined together at query time. Enforces a fixed schema and strong relational integrity (foreign keys, constraints).
- NoSQL (document/key-value/wide-column/graph): data is often denormalized — related data is embedded together so a single read gets everything without a join. Schema is flexible or absent.

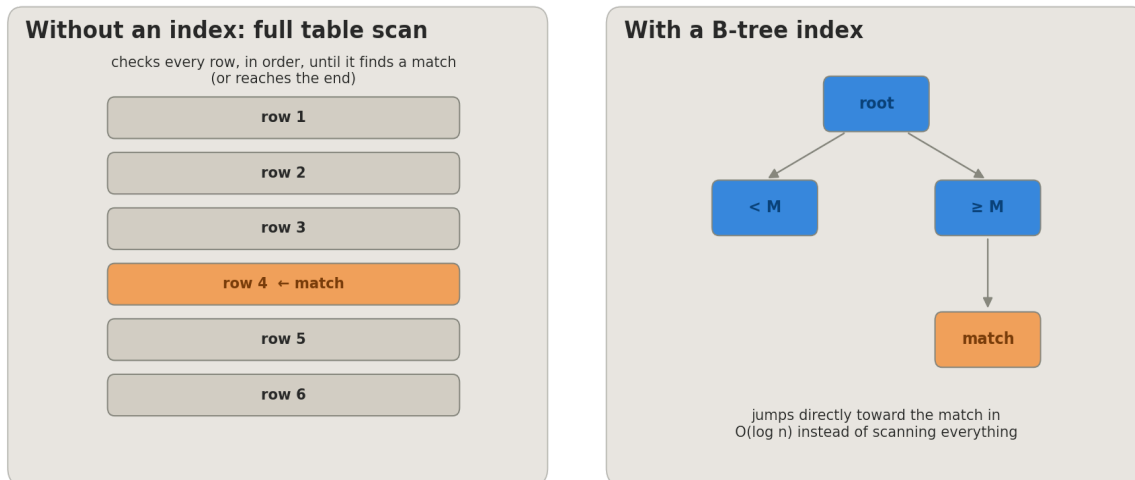
Good when (SQL): data is relational by nature, you need multi-record transactions, and strong consistency/integrity matters more than raw write throughput — e.g., financial ledgers, inventory.

Good when (NoSQL): access pattern is “fetch one object with everything about it” (a user profile, a product page), schema needs to evolve fast, or write throughput/horizontal scale matters more than relational integrity — e.g., session storage, activity feeds, IoT event logs.

The trap to avoid: picking NoSQL by default because it “scales better” — SQL databases scale horizontally too (see sharding, §4) and normalized data prevents update anomalies that denormalized data invites. The decision should follow the access pattern, not a scaling myth.

2. Indexing

An index is a separate data structure that lets the database find rows without checking every single one.



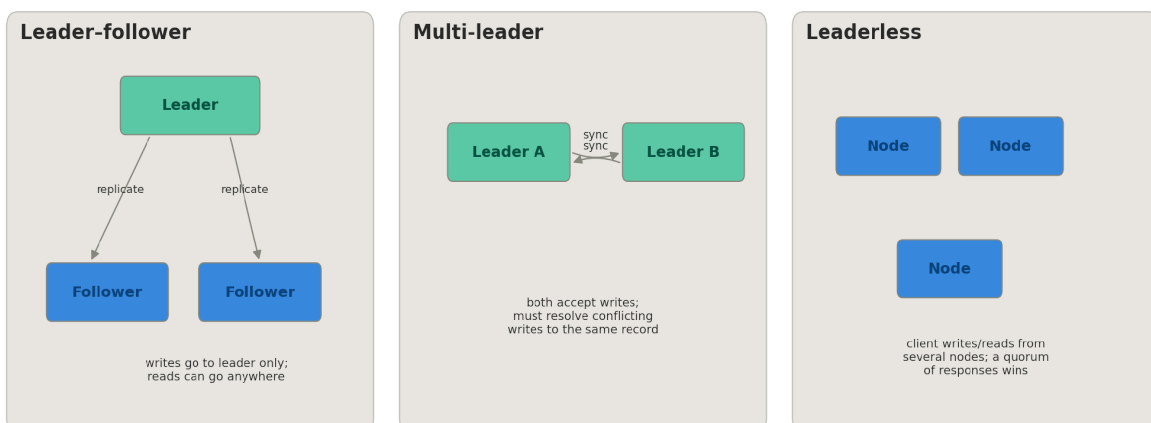
- B-tree index: the default for most relational databases. Keeps data sorted and supports $O(\log n)$ lookups, range queries, and ordered scans.
- Hash index: $O(1)$ exact-match lookups, but can't do range queries (“find everything between X and Y”) at all — the hash destroys ordering.

The cost side nobody mentions first: every index speeds up reads but slows down writes — every INSERT/UPDATE/DELETE has to also update every index on that table. A table with 10 indexes on it pays that cost 10 times per write. This is why you index the columns you actually query/filter/sort on, not every column defensively.

When an index won't help: low-cardinality columns (e.g., a boolean “is_active” flag) don't narrow the search much, and queries that need most of the table anyway may be faster with a full scan than with index lookups plus random-access reads back to the table.

3. Replication

Replication keeps multiple copies of the same data on different machines — for read scaling and for surviving a machine failure.



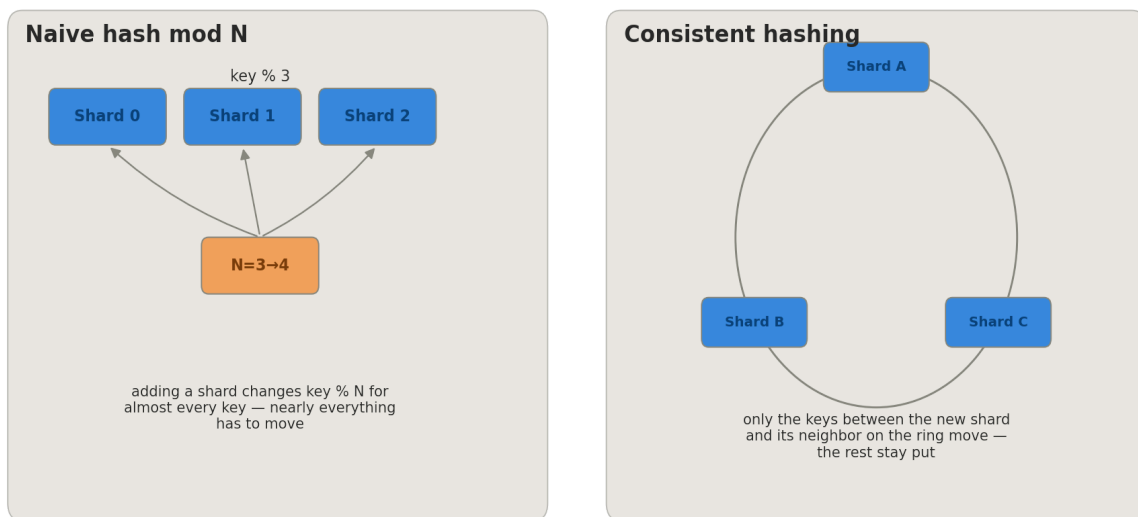
- Leader–follower (most common): all writes go through one leader, which propagates changes to followers. Followers can serve reads, spreading out read load. Simple to reason about; the leader is a single write bottleneck and a failover event if it dies.

- Multi-leader: more than one node accepts writes (often one per data center/region), and they sync with each other. Solves write availability across regions, but now two leaders can accept conflicting writes to the same record — conflict resolution becomes a real design problem.
- Leaderless: any node can accept a read or write; the client (or a coordinator) talks to several nodes and a quorum of responses determines the answer. No single point of failure for writes, at the cost of more complex consistency reasoning (this is where quorum reads/writes and read-repair come from).

Replication lag: the gap between a write landing on the leader and it showing up on a follower. This is a real failure mode, not a footnote — a user who writes then immediately reads from a lagging follower can see their own write “disappear.” This is exactly the CAP/PACELC tradeoff from Stage 1 showing up as a concrete bug.

4. Sharding / Partitioning

Sharding splits one logical dataset across multiple machines, each holding a subset of the rows — used when a single machine's storage or write throughput is the actual bottleneck, not before.



- Range-based: partition by a key range (e.g., users A–M on shard 1, N–Z on shard 2). Easy to reason about and supports range queries, but creates hot spots if the key distribution isn't uniform (e.g., alphabetically clustered signups).
- Hash-based: partition by hash(key) — spreads load evenly, but destroys range-query locality and, with naive modulo hashing, forces a massive reshuffle every time you add a shard.
- Directory-based: a separate lookup service maps each key to its shard — flexible and rebalance-friendly, but that directory is now its own critical, highly-available dependency.

Why consistent hashing exists: naive hash-mod-N sharding means adding one shard changes the target shard for almost every key, forcing nearly all data to move. Consistent hashing arranges shards on a ring so that adding or removing one shard only affects the keys immediately adjacent to it — this is the concrete mechanism, not just a buzzword.

5. ACID vs. BASE

	ACID (traditional SQL)	BASE (typical NoSQL / distributed)
Stands for	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
Guarantee	Transactions fully happen or fully don't; data is always in a valid state	System stays responsive even if some data is temporarily stale

Tradeoff	Stronger correctness, less availability/scale under partition	Higher availability/scale, weaker consistency guarantees
Good fit	Payments, inventory counts, anything where a half-applied write is unacceptable	Feed counts, view counts, caches — where staleness is a fine tradeoff for uptime

Isolation levels (the “I” in ACID, and where it actually gets nuanced)

Level	Prevents	Still allows
Read uncommitted	Nothing extra	Dirty reads (seeing another transaction's uncommitted change)
Read committed	Dirty reads	Non-repeatable reads (same query, different result, within one transaction)
Repeatable read	Non-repeatable reads	Phantom reads (new rows appearing that match your query)
Serializable	Phantom reads	Nothing — behaves as if transactions ran one at a time (at a real throughput cost)

Why this matters practically: stronger isolation is not free — it's the same latency-vs-consistency tradeoff from PACELC (Stage 1), now expressed inside a single database instead of across a network. Most production systems default to read committed and reach for stronger isolation only where a specific race condition (e.g., double-booking a seat) makes it necessary.

How this stage connects back

- Replication is what makes CAP's “partition” scenario concrete — it's the reason there are multiple copies that can disagree in the first place.
- Sharding is what you reach for only after replication and a bigger single machine stop being enough — it's a last resort, not a first move, because it adds real operational complexity (cross-shard queries, rebalancing).
- SQL/NoSQL and ACID/BASE are really the same underlying decision asked twice — how much correctness are you willing to trade for availability, throughput, or flexibility.

Must be able to explain before moving on

1. A concrete scenario where NoSQL is correct and SQL would actively hurt you, and the reverse.
2. Why every index speeds up reads but slows down writes, with the actual mechanism (updating the index structure itself).
3. Why replication lag is a real bug, not just a performance footnote — give a concrete user-facing symptom.
4. Why sharding is a last resort, and what you'd try before reaching for it.
5. Why consistent hashing exists — explain the specific problem with naive hash-mod-N sharding that it solves.